

Question 4

Q: Differentiate how pixel resolution and intensity resolution affect image representation in a practical application like medical imaging.

Theoretical Concept: Pixel vs Intensity Resolution

Pixel Resolution (Spatial): Determined by sampling. It dictates the physical details visible (e.g., identifying a tiny tumor). Higher spatial resolution = more pixels.

Intensity Resolution (Radiometric): Determined by quantization. It dictates the number of gray levels (e.g., 8-bit = 256 shades). In medical imaging (like X-rays), high intensity resolution (12-bit or 16-bit) is crucial to differentiate subtle tissue densities that look identical in 8-bit images.

OpenCV Python Solution

[Copy](#)

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to plot images

# Step 1: Create an image with a smooth, continuous gradient (high intensity resolution)
img = np.tile(np.linspace(0, 255, 256, dtype=np.uint8), (100, 1)) # Repeat an array of 256 values 100 times

# Step 2: Simulate drastically lowering the intensity resolution to just 3-bit (8 shades)
# We divide by 32, round down, and multiply by 32 to group values into 8 distinct bins
quantized = np.floor(img / 32) * 32 # Round down each pixel value to quantize into 8 shades
quantized = quantized.astype(np.uint8)

# Step 3: Display the degraded intensity image
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title('8-bit (256 shades)') # Display original image
plt.subplot(122), plt.imshow(quantized, cmap='gray'), plt.title('3-bit (8 shades)') # Display degraded image
plt.show() # Show the final plot window to the user
```

Expected Output

The output shows two gradients. The 8-bit gradient is perfectly smooth. The 3-bit gradient is broken into rigid vertical bars (false contours), demonstrating loss of intensity resolution.

How to Execute & Submit

1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.